



PLAN DE LABOR DOCENTE

Fundamentos de Programación Algoritmos y Programación I

Carreras: Ingeniería en Informática · Licenciatura en Análisis de Sistemas
Departamento de Computación

1. Programa Sintético

Práctica de programación. Buenas prácticas. Algoritmos y programas. Variables, expresiones, tipos de datos. Funciones y programación modular. Abstracción. Tipos de datos básicos y datos estructurados. Estructuras de control. Algoritmos secuenciales básicos. Algoritmos de búsqueda. Algoritmos de ordenamiento comparativos. Archivos de texto y binarios. Memoria de una computadora: punteros, pila de ejecución. Recursividad. Manejo de memoria dinámica ante la ausencia de recolección de basura automática.

2. Contenidos Analíticos

Los contenidos se presentan en orden de complejidad creciente, con desarrollo en espiral según el modelo de Bruner. Cada unidad temática integra los contenidos centrales de algoritmia y programación con el enfoque holístico transversal (arquitectura de computadoras, sistemas operativos y lenguajes de programación), de manera que la comprensión de cada nivel consolida y amplía los anteriores.

Unidad 1 — Representación de la Información y Fundamentos de Algoritmia

Historia y propiedades de los algoritmos (finitud, determinismo, efectividad). Metodología de resolución de problemas de Pólya: análisis, diseño, codificación, evaluación. Variables, expresiones y operadores en Python. Tipos de datos primitivos: enteros, flotantes, cadenas de caracteres y booleanos. Representación binaria: complemento a dos para enteros con signo y sin signo (8, 16, 32, 64 bits); estándar IEEE 754 para punto flotante; codificaciones de caracteres ASCII, UTF-8 y Unicode. Limitaciones intrínsecas (overflow, underflow, pérdida de precisión) y extrínsecas (endianness,

portabilidad). Algoritmos secuenciales: trazas y pseudocódigo. Literate programming como instrumento de documentación interna. Enfoque holístico: arquitectura Von Neumann y Harvard —CPU, ALU, Unidad de Control, registros (PC, IR, MAR, MBR), memoria unificada y bus del sistema; ciclo fetch-decode-execute; lenguaje de máquina— codificación de instrucciones (opcode, operandos, modos de direccionamiento), formatos de instrucción (3, 2, 1 y 0 direcciones).

Unidad 2 — Estructuras de Control, Funciones y Modularización

Estructuras condicionales: if / elif / else; expresiones booleanas compuestas; cortocircuito lógico. Estructuras iterativas: while, for, range(); iteración sobre secuencias. Diseño de casos de prueba: escenarios normales, límite y extremos. Funciones: definición, parámetros por valor y por referencia, valores de retorno, alcance de variables (scope). Programación modular: descomposición top-down, cohesión, acoplamiento. Abstracción funcional y contratos de interfaz. Enfoque holístico: memoria operativa vs. almacenamiento secundario; sistema operativo como gestor de recursos — rol del kernel, llamadas al sistema (syscalls), gestión de procesos (estados: nuevo, listo, en ejecución, bloqueado, terminado; Process Control Block — PCB); pila de ejecución (stack frames, dirección de retorno, variables locales); compiladores vs. intérpretes — fases de compilación (análisis léxico, sintáctico, semántico, generación de código); Python como lenguaje interpretado: bytecode, máquina virtual CPython, módulo dis.

Unidad 3 — Algoritmos de Búsqueda, Ordenamiento y Archivos

Algoritmos de búsqueda: búsqueda lineal $O(n)$ y búsqueda binaria $O(\log n)$ — iterativa y recursiva. Introducción a la complejidad algorítmica: notación asintótica O ; análisis de casos promedio y peor. Algoritmos de ordenamiento comparativos: selección, inserción y burbuja ($O(n^2)$); merge sort y quick sort ($O(n \log n)$) — estrategia divide y vencerás, análisis de pivotes, casos degenerados. Comparación empírica de rendimiento mediante profiling (timeit, cProfile). Archivos de texto en Python: apertura, lectura, escritura, cierre, context managers (with). Archivos binarios: acceso con el módulo struct; serialización de datos. Enfoque holístico: jerarquía de memoria — registros, caché L1/L2, RAM, disco; principio de localidad de referencia; sistema de archivos del SO — organización (árbol de directorios), metadatos (inodos), permisos, operaciones de E/S; planificación de procesos — FIFO, SJF, Round-Robin.

Unidad 4 — Punteros, Recursividad y Manejo de Memoria Dinámica

Modelo de memoria de un programa: segmentos text, data, stack y heap. Referencias y punteros en Python: identidad de objetos (id()), mutabilidad, aliasing y sus consecuencias. Paso de argumentos: por valor y por referencia; efectos secundarios. Recursividad: estructura (caso base, caso recursivo), árbol de llamadas, correctitud. Comparación sistemática iteración vs. recursión: legibilidad, profundidad de pila, eficiencia. Recursividad de cola (tail recursion) y sus limitaciones en Python. Manejo de memoria dinámica: heap, fragmentación, ausencia de recolección de basura automática en C/C++; garbage collector en Python (reference counting, ciclos). Enfoque holístico: gestión de memoria en el SO — memoria virtual, paginación, segmentación, TLB; jerarquía de almacenamiento; gramáticas formales — gramáticas libres de contexto, árboles de derivación, relación con la sintaxis de lenguajes de programación (BNF, EBNF).

Unidad 5 — Estructuras de Datos Complejas y Algoritmos Avanzados

Tipos de datos estructurados en Python: list, tuple, dict, set — representación interna en memoria (arrays dinámicos, tablas hash), complejidad de operaciones (acceso, búsqueda, inserción, eliminación). Listas de listas (matrices): indexación, recorrido, aplicaciones. Comprensiones de listas y diccionarios: expresividad y eficiencia. Generadores: yield, iteradores perezosos, eficiencia en memoria. Tipos Abstractos de Datos (TAD): concepto, interfaz vs. implementación. Patrones algorítmicos iterativos avanzados: two-pointer, sliding window, conteo y acumulación; trade-offs tiempo-espacio. Algoritmos recursivos avanzados: backtracking — estructura general, poda, aplicaciones (permutaciones,

subconjuntos, problemas de satisfacción de restricciones); profundización en divide y vencerás. Integración de algoritmos y estructuras en problemas complejos. Enfoque holístico: abstracción de procesos en el SO — threads y procesos, concurrencia básica; Python internals avanzado: GIL, multiprocessing; virtualización conceptual — máquinas virtuales y contenedores (Docker); impacto de la arquitectura en la eficiencia algorítmica; comparativa de arquitecturas Von Neumann vs. Harvard en contextos reales (DSPs, microcontroladores). Uso crítico y ético de la inteligencia artificial generativa en la programación.

3. Objetivos

3.1 Marco Curricular Institucional

El Marco Curricular de la FIUBA establece que los graduados deberán poseer: formación académica científica y tecnológica sólida y actualizada; competencia para seleccionar y utilizar efectivamente técnicas y herramientas de la disciplina; capacidad para diseñar, planificar, realizar, evaluar, mejorar y gestionar proyectos y soluciones a problemas profesionales complejos; formación integral que habilite el ejercicio profesional con perspectiva interdisciplinaria, ética y compromiso social; competencias creativas, emprendedoras y de liderazgo de equipos diversos; capacidad de aprendizaje continuo y autónomo; y competencias comunicacionales en contextos interdisciplinarios e internacionales, incluyendo el uso del español y el inglés.

3.2 Objetivos de Enseñanza

Generales

- Desarrollar pensamiento algorítmico y habilidades de programación mediante metodologías activas: clase invertida, aprendizaje basado en problemas, pensamiento de diseño, aprendizaje basado en proyectos y trabajo colaborativo.
- Fomentar el uso crítico, ético y creativo de herramientas de IA generativa, promoviendo la reflexión sobre su alcance y limitaciones.
- Prevenir el sedentarismo cognitivo mediante la resolución de problemas reales que demandan reflexión y diseño autónomo.
- Familiarizar al estudiante con el modo de pensar en ingeniería: resolución sistemática, disciplinada y cuantificable de problemas para obtener soluciones eficientes.

Específicos

- Que el estudiante tome conciencia de la importancia de la algoritmia como paradigma de resolución de problemas y de la programación como práctica de resolución de problemas con la computadora.
- Que el estudiante desarrolle la capacidad de relacionar esquemas de solución de problemas con la resolución de problemas algorítmicos, con énfasis en el método científico (Pólya).
- Que el estudiante desarrolle la capacidad de análisis, sistematización, programación y procesamiento de problemas de tipo técnico-científicos, aplicables en su formación académica y ejercicio profesional.

3.3 Objetivos de Aprendizaje

Los objetivos de aprendizaje se organizan en torno a las competencias de ingeniería definidas a continuación y se operacionalizan a través del sistema de códigos mnemónicos presentado en la Sección 4.

4. Competencias de Aprendizaje con Códigos Mnemónicos

Las competencias se clasifican en cuatro categorías: Cognitivas (CC), Procedimentales (CP), Actitudinales (CA), Transversales (CT) y Generales de Ingeniería (CG). Los códigos se utilizan en el Desarrollo de Contenidos para identificar las competencias trabajadas en cada instancia de enseñanza y aprendizaje.

Código	Categoría	Descripción de la Competencia
CC1	Cognitiva	Conocimiento general sobre algoritmia; uso y programación de computadoras; sintaxis de Python.
CC2	Cognitiva	Conocimiento de organización, capacidades y funcionamiento de las computadoras (arquitectura, SO) y fundamentos de su programación.
CC3	Cognitiva	Conocimiento y aplicación de procedimientos algorítmicos básicos; diseño de soluciones algorítmicas eficientes.
CC4	Cognitiva	Conocimiento de tipos y estructuras de datos fundamentales y su aplicación apropiada para la resolución eficiente de problemas.
CP1	Procedimental	Capacidad de resolución de problemas mediante algoritmos y programas ejecutables en Python (programa eficaz).
CP2	Procedimental	Capacidad para descomponer un problema real en subproblemas y modularizar su solución.
CP3	Procedimental	Capacidad para documentar programas con claridad (<i>literate programming</i>) y comprender documentación técnica de terceros.
CA1	Actitudinal	Motivación por la claridad, sencillez, eficacia y eficiencia algorítmica.
CA2	Actitudinal	Capacidad de debatir y concluir sobre distintas soluciones algorítmicas y sus traducciones a programas.
CT1	Transversal	Capacidad para la autoorganización y planificación del trabajo individual y del proceso de aprendizaje.
CT2	Transversal	Capacidad para el trabajo en grupo: colaboración, distribución de tareas, integración de aportes.
CT3	Transversal	Capacidad de análisis y síntesis: identificar estructuras, patrones y relaciones en los problemas.
CT4	Transversal	Motivación por la calidad del resultado: revisión, refinamiento y mejora continua de soluciones.
CT5	Transversal	Disposición para el compromiso, la responsabilidad, la honestidad y el respeto en el trabajo individual y grupal.
CG1	Ingeniería General	Conocimiento de tecnologías y métodos básicos; versatilidad para adaptarse a nuevas situaciones y herramientas.
CG2	Ingeniería General	Capacidad de resolver problemas con iniciativa, toma de decisiones, creatividad y sentido crítico.
CG3	Ingeniería General	Capacidad de comunicar y transmitir conocimientos, habilidades y destrezas; responsabilidad profesional.

Nota: los códigos aparecen en las tablas semanales de Desarrollo de Contenidos en la fila 'Competencias a desarrollar'. Permiten trazar la cobertura de competencias a lo largo del cursado y facilitar la alineación constructiva entre objetivos, actividades y evaluación.

5. Visión General y Fundamentación

Este plan de labor docente contempla dieciséis semanas de dos clases por semana, una teórica y una práctica en laboratorio de tres horas cada una, más seis horas de actividades formativas de extracursado, centradas en Algoritmia y Programación con un enfoque holístico que articula conocimientos transversales (Arquitectura de Computadoras y Sistemas Operativos) y verticales (Lenguajes, Compiladores e Interpretes).

El diseño curricular se fundamenta en la Espiral de Bruner: los temas se revisan de manera repetida y con complejidad creciente, reforzando el aprendizaje a través de la interacción y la experiencia. Cada contenido se introduce de manera elemental en su primera aparición y se profundiza progresivamente en ciclos posteriores, siempre en diálogo con los saberes específicos de algoritmos y programación. El propósito es favorecer una comprensión estructural de la informática donde el desarrollo técnico se apoye en una perspectiva integrada y crítica que privilegie la eficiencia sobre la mera eficacia.

Las últimas cuatro semanas (13 a 16) se reservan íntegramente para el desarrollo, seguimiento y defensa del proyecto final integrador, articulando todos los saberes construidos durante el cursado.

5.1 Metodología y Estrategias

- Clase invertida: antes de la clase presencial, los estudiantes acceden a videos, lecturas y simulaciones; en el aula se dedica el tiempo a la resolución colaborativa de problemas, discusión de errores y análisis crítico de soluciones generadas por IA; después de la clase se realiza una reflexión escrita sobre el proceso cognitivo implicado.
- Modelo espiral de Bruner: cada semana se introducen nociones básicas de Arquitectura, SO y Lenguajes vinculadas con los temas principales de Algoritmia y Programación, y se retoman y expanden en ciclos posteriores, conectando teoría y aplicación.
- Aprendizaje basado en problemas (ABP): se resuelven desafíos que demandan el uso simultáneo de conceptos estructurales (memoria, procesos, compilación) y habilidades algorítmicas.
- Resolución de problemas con metodología Pólya: análisis (casos de análisis y prueba), diseño (pseudocódigo / literate programming), codificación (implementación en Python) y evaluación (verificación con los casos definidos).
- Evaluación continua formativa: se monitorean avances mediante ejercicios prácticos semanales, autoevaluaciones y foros de discusión críticos sobre el uso de herramientas digitales e IA generativa.
- Proyecto final colaborativo: las semanas 13 a 16 se orientan al diseño, desarrollo, optimización, presentación y defensa de una solución concreta que integra todos los saberes.
- Uso reflexivo y ético de IA generativa: los estudiantes analizan críticamente los resultados producidos por IA, evitan la repetición mecánica y documentan sus interacciones con estas herramientas como parte de la evaluación.
- Repositorio colaborativo en GitHub: el historial de commits forma parte del proceso evaluativo, evidenciando la evolución del pensamiento algorítmico y el trabajo en equipo.

6. Desarrollo de Contenidos

Para cada semana se detallan los temas y actividades de la clase teórica (3 horas), la clase práctica en laboratorio (3 horas) y las actividades formativas de extracursado (6 horas aproximadas). La columna de competencias asocia los códigos mnemónicos definidos en la Sección 4. Los temas holísticos (Arquitectura, SO, Lenguajes) se integran en la clase teórica como contexto de comprensión profunda de los algoritmos y programas desarrollados en laboratorio.

6.1 Semanas 1–12: Contenidos del Cursado

Semana 1 Variables, tipos de datos y algoritmos secuenciales (I)	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
- Introducción a la algoritmia: propiedades, historia (Euclides, Al-Hwarizmi), importancia en la ingeniería. - Metodología Pólya aplicada a programación: análisis, diseño, codificación, evaluación. - Variables, expresiones y tipos de datos primitivos: int, float, str, bool. [Holístico] Arquitectura Von Neumann: CPU (ALU, UC, registros), memoria unificada, bus del sistema, ciclo fetch-decode-execute.	- Configuración del entorno de trabajo. - Primeros programas en Python: entrada/salida, asignación, operaciones aritméticas y de cadenas. - Traza manual de algoritmos secuenciales simples. Comparación con salida del intérprete. - Identificación de errores de sintaxis vs. semántica.
Actividades de Extracursado (6 h aprox.)	
- Lectura del documento 'Arquitecturas de Computadoras: Von Neumann vs Harvard'. - Ejercicios de traza manual de algoritmos secuenciales (plataforma Moodle). - Reflexión escrita (foro): ¿Cómo repercute el progreso de la informática en los principios y la organización de la sociedad?	
Competencias a desarrollar: CC1 · CC2 · CP1 · CT3 · CG1	

Semana 2 Variables, tipos de datos y algoritmos secuenciales (II)	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
- Tipos de datos básicos en profundidad: representación binaria de enteros (complemento a dos), flotantes (IEEE 754), caracteres (ASCII, Unicode/UTF-8) y booleanos. - Limitaciones intrínsecas (overflow, underflow, precisión) y extrínsecas (endianness, portabilidad). - Buenas prácticas de programación: nombrado, comentarios, literate programming. [Holístico] Lenguaje de máquina: codificación de instrucciones (opcode, operandos), modos de direccionamiento, relación entre dato y representación física.	- Ejercicios con tipos numéricos: desbordamiento, precisión flotante, conversiones explícitas. - Documentación interna (literate programming): descripción del algoritmo antes de codificar. - Análisis crítico de código generado por IA generativa: correctitud, estilo, eficiencia. - Uso del depurador básico de Python para inspeccionar variables.
Actividades de Extracursado (6 h aprox.)	
- Lectura del documento 'Tipos de Datos Básicos en Computadoras'. - Lectura del documento 'Lenguajes de Máquina y Evolución de la Programación'. - Ejercicios de representación binaria y conversión de tipos. - Autoevaluación (Moodle): tipos de datos y buenas prácticas.	
Competencias a desarrollar: CC1 · CC2 · CC3 · CP1 · CP3 · CA1 · CT3	

Semana 3 Estructuras de control (I): condicionales e iteraciones	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
Estructuras condicionales: if / elif / else; expresiones booleanas compuestas, cortocircuito lógico.	Resolución de problemas con condicionales múltiples y bucles: validación de datos, cálculo iterativo.

• Estructuras iterativas: while, for, range(); iteración sobre secuencias. • Diseño de casos de prueba: escenarios normales, límite y extremos (casos de análisis). [Holístico] Saltos condicionales e incondicionales en lenguaje de máquina; memoria operativa vs. almacenamiento secundario; rol del SO como gestor de recursos.	• Diseño explícito de casos de análisis antes de codificar (fase de análisis de Pólya). • Discusión colaborativa de errores comunes: bucles infinitos, condiciones erróneas. • Primeros ejercicios de análisis de complejidad intuitiva.
Actividades de Extracursado (6 h aprox.)	
• Video: estructuras de control y su traducción a código máquina. • Ejercicios en plataforma (Moodle): condicionales y bucles con casos límite. • Foro: reflexión sobre la importancia del diseño de casos de prueba. • Lectura introductoria del documento 'Funciones del Sistema Operativo'.	
Competencias a desarrollar: CC1 · CC3 · CP1 · CP2 · CA1 · CT3 · CG2	

Semana 4 Funciones, modularización y abstracción	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Funciones: definición, parámetros, valores de retorno, alcance de variables (scope). • Programación modular: descomposición de problemas; criterios de cohesión y acoplamiento. • Abstracción funcional: diseño top-down, contratos de interfaz. [Holístico] Pila de ejecución (stack frames): introducción; compiladores e intérpretes (primeras diferencias); Python como lenguaje interpretado (bytecode, CPython).	• Refactorización de soluciones previas usando funciones (modularización). • Documentación interna completa: literate programming aplicada a funciones. • Ejercicios de diseño modular: dividir un problema complejo en funciones colaborativas. • Inspección del stack trace de Python ante errores de recursión.
Actividades de Extracursado (6 h aprox.)	
• Lectura del documento 'Fundamentos de Programación: Algoritmia y Python'. • Ejercicios de modularización en Moodle. • Autoevaluación: alcance de variables, parámetros vs. argumentos. • Reflexión escrita: ¿cuándo conviene abstraer una función? Criterios de diseño.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CP3 · CA1 · CA2 · CT1 · CG2	

Semana 5 Algoritmos de búsqueda y complejidad algorítmica	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Búsqueda lineal y búsqueda binaria: diseño, correctitud, análisis comparativo. • Introducción a la complejidad algorítmica: notación $O(n)$ y $O(\log n)$; análisis intuitivo de casos promedio y peor. • Precondición de la búsqueda binaria: secuencias ordenadas. [Holístico] Jerarquía de memoria (registros, caché L1/L2, RAM, disco): impacto en la eficiencia de acceso.	• Implementación y comparación experimental de búsqueda lineal vs. binaria (tiempos de ejecución con `time`). • Análisis de búsqueda binaria recursiva e iterativa. • Diseño de casos de prueba para validar correctitud de la búsqueda. • Análisis crítico de implementaciones generadas por IA: eficiencia y correctitud.

Administración de archivos en el SO: sistema de ficheros, rutas, permisos.	
Actividades de Extracursado (6 h aprox.)	
• Ejercicios de análisis de complejidad intuitiva en Moodle. • Video: jerarquía de memoria y localidad de referencia. • Lectura: sistemas de archivos en el SO (sección del documento 'Funciones del SO'). • Foro: debate sobre eficiencia vs. legibilidad del código.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CA1 · CT3 · CG2	

Semana 6 Algoritmos de ordenamiento y archivos de texto	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Algoritmos de ordenamiento comparativos: selección, inserción, burbuja — diseño, correctitud y complejidad $O(n^2)$. • Merge sort: estrategia divide y vencerás, complejidad $O(n \log n)$; introducción a quick sort. • Archivos de texto en Python: apertura, lectura, escritura, cierre; context managers (with). [Holístico] Compiladores vs. intérpretes: fases de compilación (análisis léxico, sintáctico, semántico, generación de código). Python bytecode e intérprete CPython. Procesos del SO y su ciclo de vida.	• Implementación de al menos tres algoritmos de ordenamiento; comparación experimental de rendimiento. • Procesamiento de archivos de texto: lectura de datos estructurados, conteo, transformaciones. • Visualización de pasos de ordenamiento (trazas). • Discusión colaborativa: ¿cuándo elegir cada algoritmo?
Actividades de Extracursado (6 h aprox.)	
• Ejercicios combinados: leer archivo, ordenar datos, escribir resultado. • Autoevaluación: ordenamiento y archivos. • Reflexión escrita: análisis del propio proceso de aprendizaje en las semanas 1–6.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CP3 · CA1 · CA2 · CT1 · CT3	

Semana 7 Punteros, referencias y modelo de memoria	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Modelo de memoria de un programa: segmentos text, data, stack y heap. • Referencias y punteros en Python: identidad de objetos (<code>id()</code>), mutabilidad, aliasing. • Paso de argumentos por valor y por referencia; efectos secundarios. [Holístico] Gestión de memoria en el SO: memoria virtual, paginación, segmentación. Pila de ejecución del sistema: stack frames, dirección de retorno, variables locales.	• Visualización de referencias con Python Tutor: objetos mutables e inmutables. • Experimentos con aliasing: listas mutables vs. tuplas inmutables. • Primeros algoritmos recursivos: factorial, suma de lista, potencia. • Análisis de uso de memoria con <code>sys.getsizeof()</code>.
Actividades de Extracursado (6 h aprox.)	
• Lectura: gestión de memoria en el SO (sección del documento 'Funciones del SO'). • Ejercicios con referencias y aliasing en Moodle. • Simulador interactivo de pila de ejecución (recurso online).	

- Foro: diferencias entre paso por valor y por referencia en distintos lenguajes.

Competencias a desarrollar: CC2 · CC3 · CC4 · CP1 · CP2 · CT3 · CG1

Semana 8 Recursividad y manejo de memoria dinámica

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Recursividad: estructura (caso base, caso recursivo), correctitud, árbol de recursión. • Comparación sistemática iteración vs. recursión: legibilidad, uso de stack, eficiencia. • Recursividad de cola (tail recursion) y sus limitaciones en Python. • Manejo de memoria dinámica: heap, ausencia de recolección de basura automática en C/C++; garbage collector en Python. [Holístico] Gramáticas formales básicas: gramáticas libres de contexto, árboles de derivación, relación con la sintaxis de lenguajes de programación.	• Torres de Hanói: diseño recursivo, visualización del árbol de llamadas. • Búsqueda binaria recursiva vs. iterativa: comparación de rendimiento y legibilidad. • Archivos binarios en Python: lectura y escritura con `struct`; concepto de serialización. • Análisis de `RecursionError` en Python: relación con la pila del SO.
Actividades de Extracursado (6 h aprox.)	
• Ejercicios de recursividad (Moodle): problemas clásicos con análisis de casos. • Lectura: gramáticas y lenguajes formales (sección del documento 'Lenguajes de Máquina'). • Autoevaluación: recursividad y memoria dinámica. • Reflexión escrita: ¿en qué problemas prefiero recursión y por qué?	
Competencias a desarrollar: CC2 · CC3 · CP1 · CP2 · CA1 · CA2 · CT3 · CG2	

Semana 9 Estructuras de datos: listas, tuplas, diccionarios y conjuntos

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Tipos de datos estructurados en Python: list, tuple, dict, set — representación en memoria, complejidad de operaciones. • Listas de listas (matrices): indexación, recorrido, aplicaciones. • Comprensiones de listas y diccionarios: expresividad y eficiencia. [Holístico] Tipos Abstractos de Datos (TAD): concepto, interfaz vs. implementación. Primer contacto con abstracción de procesos en el SO: threads y procesos.	• Resolución de problemas usando estructuras de datos adecuadas: histogramas, matrices de adyacencia, tablas de frecuencia. • Comparación de complejidad: lista vs. diccionario para búsqueda y acceso. • Ejercicios de comprensiones de listas: transformación y filtrado. • Análisis crítico de soluciones propuestas por IA.
Actividades de Extracursado (6 h aprox.)	
• Lectura: TAD y estructuras de datos (material complementario del campus). • Ejercicios con estructuras de datos en Moodle. • Video: cómo Python implementa listas y diccionarios internamente (hash tables). • Foro: debate sobre cuándo usar cada estructura de datos.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CA1 · CT2 · CT3 · CG2	

Semana 10 Algoritmos iterativos avanzados y eficiencia

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Patrones algorítmicos iterativos: two-pointer, sliding window, conteo y acumulación. • Complejidad espacial $O(1)$ vs. $O(n)$: trade-offs tiempo-espacio. • Generadores en Python: `yield`, iteradores perezosos; eficiencia en memoria. [Holístico] Python internals: compilación a bytecode, máquina virtual CPython, módulo `dis`; cómo el SO planifica procesos (scheduling básico): FIFO, SJF, Round-Robin.	• Implementación de algoritmos two-pointer y sliding window sobre listas. • Uso de generadores para procesar archivos de gran tamaño. • Profiling básico con `cProfile` y `timeit`: medición empírica de rendimiento. • Comparación de soluciones iterativas equivalentes con distintas complejidades.
Actividades de Extracursado (6 h aprox.)	
• Ejercicios de patrones iterativos en Moodle. • Lectura: planificación de procesos en el SO (documento del proyecto). • Autoevaluación: complejidad temporal y espacial. • Ejercicio de optimización: mejorar una solución $O(n^2)$ a $O(n)$.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CA1 · CT1 · CT3 · CG2	

Semana 11 Algoritmos recursivos avanzados: backtracking y divide y vencerás

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Divide y vencerás: paradigma, merge sort profundizado, quick sort con análisis de pivote. • Backtracking: estructura general, poda, aplicaciones (permutaciones, subconjuntos, laberintos). • Comparación experimental de algoritmos de ordenamiento: $O(n^2)$ vs. $O(n \log n)$. [Holístico] Paralelismo y concurrencia: introducción conceptual; relación entre divide y vencerás y cómputo paralelo. Virtualización básica: máquinas virtuales y contenedores (concepto).	• Implementación y benchmarking de merge sort vs. quick sort vs. sorted(). • Algoritmo de backtracking: generación de permutaciones y solución de laberinto simple. • Visualización del árbol de recursión de quick sort con distintos pivotes. • Discusión colaborativa: análisis crítico del uso de IA en la resolución de los problemas del día.
Actividades de Extracursado (6 h aprox.)	
• Ejercicios de backtracking y divide y vencerás en Moodle. • Reflexión escrita: comparación de paradigmas recursivos (profundidad, amplitud, backtracking). • Inicio de la identificación del problema para el proyecto final.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CA1 · CA2 · CT2 · CT3 · CG2	

Semana 12 Integración de contenidos y preparación del proyecto final

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Integración: estructuras de datos + algoritmos de búsqueda, ordenamiento, recursividad. • Análisis crítico del uso de IA generativa: casos de uso, limitaciones, responsabilidad intelectual y ética. • Introducción al proyecto final: criterios de selección de problemas, metodología de trabajo colaborativo, rubrica de evaluación.	• Resolución de un problema integrador complejo que demanda múltiples estructuras y algoritmos. • Análisis de distintas soluciones: comparación de complejidad, legibilidad y mantenibilidad. • Formación de equipos para el proyecto final; selección y planificación preliminar del problema.

[Holístico] Visión de conjunto del sistema computacional: hardware → SO → lenguaje → programa. Impacto de la arquitectura en la eficiencia del algoritmo. Comparativa Von Neumann vs. Harvard.	• Práctica de presentación oral breve de una solución algorítmica.
Actividades de Extracursado (6 h aprox.)	
- Entrega del plan del proyecto final: descripción del problema, casos de análisis, estrategia de diseño. - Autoevaluación integral de las competencias desarrolladas (semanas 1–12). - Foro de reflexión: 'La IA generativa en mi aprendizaje de programación: ventajas, riesgos y estrategias'. - Revisión de la rúbrica de evaluación del proyecto final.	
Competencias a desarrollar: CC1 · CC2 · CC3 · CC4 · CP1 · CP2 · CP3 · CA1 · CA2 · CT1 · CT2 · CT3 · CT4 · CG2 · CG3	

6.2 Semanas 13–16: Proyecto Final Integrador

Las últimas cuatro semanas se reservan íntegramente para el desarrollo del proyecto final con seguimiento semanal por parte del docente y presentación y defensa oral en la Semana 16. El proyecto es de carácter colaborativo (equipos de 2 a 3 integrantes), con seguimiento individual de contribuciones vía GitHub. Cada semana tiene una orientación específica que sigue las fases de Pólya a escala de proyecto.

Semana 13 Proyecto Final — Fase 1: Análisis y diseño del algoritmo	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
- Presentación y validación de los problemas de cada equipo con el docente. - Taller de análisis profundo: construcción de casos de análisis exhaustivos (escenarios normales, límite y extremos). - Estrategias de diseño algorítmico: elección de estructuras de datos y algoritmos justificada. - Revisión de criterios de evaluación: pensamiento algorítmico, modularización, documentación interna, ética en el uso de IA.	- Trabajo en equipo: diseño del algoritmo en pseudocódigo / literate programming. - Revisión entre pares (peer review) del diseño de cada equipo. - Consulta individual o grupal con el docente sobre el diseño propuesto. - Primer prototipo: esquema de funciones y módulos.
Actividades de Extracursado (6 h aprox.)	
- Entrega del diseño completo: pseudocódigo o literate programming documentado. - Avance de codificación del módulo principal. - Registro de interacciones con IA generativa: prompts usados, código obtenido, análisis crítico realizado. - Lectura de documentación técnica relevante al dominio del problema.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CP3 · CA1 · CT1 · CT2 · CT4 · CG2 · CG3	

Semana 14 Proyecto Final — Fase 2: Codificación e integración	
Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
- Taller de codificación: buenas prácticas, refactorización, manejo de errores ('try/except'). - Integración de módulos: pruebas de integración, detección y resolución de conflictos.	- Sesión de codificación colaborativa: integración de todos los módulos del proyecto. - Pruebas con los casos de análisis definidos en la fase 1.

• Seguimiento docente: revisión del avance de cada equipo, orientación técnica. • Uso ético y reflexivo de IA generativa en la etapa de codificación: análisis de código generado.	• Corrección de errores y optimización inicial. • Documentación interna completa: comentarios descriptivos, docstrings.
Actividades de Extracursado (6 h aprox.)	
• Entrega del código integrado con documentación interna. • Ejecución de todos los casos de análisis y registro de resultados. • Preparación del análisis del uso de IA generativa para el informe final. • Inicio de la preparación de la presentación oral.	
Competencias a desarrollar: CP1 · CP2 · CP3 · CA1 · CA2 · CT1 · CT2 · CT4 · CG2 · CG3	

Semana 15 Proyecto Final — Fase 3: Evaluación, optimización y preparación de la defensa

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Taller de evaluación: análisis de eficiencia (complejidad temporal y espacial del proyecto). • Estrategias de optimización: identificación de cuellos de botella con profiling. • Preparación de la defensa oral: estructura de la presentación, argumentación de decisiones algorítmicas. • Revisión final del análisis crítico del uso de IA generativa.	• Sesión de presentaciones preliminares (rehearsal): feedback entre pares y del docente. • Optimización guiada: al menos una mejora algorítmica documentada y justificada. • Revisión final del código: consistencia de estilo, documentación, manejo de errores. • Ensayo de respuestas a preguntas conceptuales sobre computadoras, datos y SO.
Actividades de Extracursado (6 h aprox.)	
• Entrega del código final optimizado. • Entrega del informe escrito: descripción del problema, diseño, decisiones, análisis de complejidad, análisis crítico del uso de IA. • Preparación final de la presentación (slides o Jupyter Notebook). • Autoevaluación y coevaluación de la contribución de cada integrante al proyecto.	
Competencias a desarrollar: CC3 · CC4 · CP1 · CP2 · CP3 · CA1 · CA2 · CT1 · CT2 · CT4 · CT5 · CG2 · CG3	

Semana 16 Proyecto Final — Fase 4: Presentación y defensa oral

Clase Teórica (3 h)	Clase Práctica en Laboratorio (3 h)
• Presentaciones finales de todos los equipos (15–20 min por equipo): exposición del problema, diseño, implementación y resultados. • Defensa oral: respuesta a preguntas del docente y pares sobre decisiones algorítmicas, estructurales y de diseño. • Evaluación conjunta: rúbrica de resolución de problemas + criterios del proyecto final.	• Cierre de la asignatura: reflexión colectiva sobre el recorrido de aprendizaje. • Debate final: 'Algoritmia y programación en la era de la IA generativa: desafíos y oportunidades para el ingeniero'. • Entrega final de autoevaluación y coevaluación.
Actividades de Extracursado (6 h aprox.)	
• Cierre del repositorio en GitHub: código final, README, historial de commits. • Encuesta de satisfacción y reflexión final sobre el proceso de aprendizaje.	

Competencias a desarrollar: CC1 · CC2 · CC3 · CC4 · CP1 · CP2 · CP3 · CA1
 · CA2 · CT1 · CT2 · CT3 · CT4 · CT5 · CG1 · CG2 · CG3

7. Sistema de Evaluación

7.1 Modalidades de Evaluación del Cursado

- Evaluación formativa continua (Semanas 1–12): ejercicios semanales alineados con los contenidos espiralados. Se prioriza el proceso sobre el resultado, valorando cualitativamente la evolución del pensamiento algorítmico mediante la rúbrica de resolución de problemas.
- Autoevaluaciones y coevaluaciones (Moodle): reflexión sobre el propio proceso de aprendizaje y la relación entre saberes específicos y transversales.
- Foros de discusión crítica: reflexión colectiva sobre el uso de IA generativa, decisiones algorítmicas y relaciones entre contenidos del cursado.
- Proyecto final integrador (Semanas 13–16): desarrollo de una solución colaborativa que expone la integración entre algoritmia, programación y conceptos holísticos. Incluye informe escrito con análisis crítico del uso de IA generativa, código en Python con documentación interna completa (literate programming), repositorio en GitHub y presentación oral con defensa.
- Evaluación integradora individual: resolución de un programa en Python y respuesta a planteos conceptuales o procedimentales sobre computadoras, datos y sistemas operativos.

7.2 Métricas de Evaluación del Cursado

Criterio	Instrumento de Evaluación	Peso
Pensamiento algorítmico	Rúbrica de resolución de problemas (ejercicios semanales)	30 %
Claridad y documentación de programas	Evaluación de documentación interna (literate programming)	20 %
Participación activa	Autoevaluación + coevaluación (foros, debates, peer review)	20 %
Proyecto final integrador	Presentación + defensa oral + informe escrito	30 %

7.3 Rúbrica de Resolución de Problemas

La rúbrica valora la evolución del pensamiento algorítmico en ejercicios semanales y en el proyecto final, priorizando el proceso de aprendizaje, la reflexión crítica y la integración de saberes. Los criterios incorporan los códigos de competencias asociados (entre paréntesis) para facilitar el seguimiento y la retroalimentación personalizada.

Criterio	Nivel Básico	Nivel Intermedio	Nivel Avanzado	Nivel Experto
Comprensión del Problema (CC1–CC3)	Identifica parcialmente el problema, con errores o confusiones.	Reconoce el problema y sus componentes principales.	Analiza detalladamente el problema, identificando relaciones y restricciones.	Plantea preguntas críticas y redefine el problema en función del contexto.
Diseño Algorítmico (CC3–CC4, CP1–CP2)	Propone una secuencia de pasos	Elabora algoritmos correctos para casos estándar.	Utiliza estructuras de control y datos	Optimiza el diseño, justifica elecciones y

	simple, con posibles omisiones.		adecuados; anticipa casos extremos.	propone mejoras innovadoras.
Abstracción y Modularización (CP2, CA1)	Resuelve el problema sin separar en módulos o funciones claras.	Divide la solución en módulos básicos.	Aplica abstracciones efectivas y reutiliza componentes.	Integra múltiples abstracciones, facilitando escalabilidad y mantenimiento.
Iteración y Recursividad (CC3, CP1)	Utiliza bucles o recursión solo en casos simples o guiados.	Resuelve problemas mediante iteración o recursión, con apoyo.	Selecciona y aplica apropiadamente entre iteración y recursión.	Combina iteración y recursividad, optimizando recursos y legibilidad.
Documentación (CP3, CA1)	Ausencia de comentarios o documentación mínima e incorrecta.	Comentarios básicos que describen el qué pero no el por qué.	Literate programming aplicado: describe el proceso y las decisiones.	Documentación completa; el código es autoexplicativo e incluye reflexiones críticas.
Evaluación y Mejora (CC3, CA2)	Rara vez revisa o mejora la solución propuesta.	Detecta y corrige errores básicos en la solución.	Analiza eficiencia y busca optimizar recursos.	Realiza análisis crítico, compara alternativas y documenta reflexiones.
Colaboración y Comunicación (CT2, CG3)	Explica ideas de manera limitada y poco clara.	Comunica soluciones con ejemplos simples.	Argumenta y defiende decisiones algorítmicas en equipo.	Guía, integra aportes y fomenta el debate crítico en el grupo.

8. Recursos y Herramientas

- Campus Virtual FIUBA (Moodle): publicación de material académico, planteo y formalización de entregas, evaluación de problemas, foros de reflexión, autoevaluaciones, encuestas y consultas asincrónicas.
- Python 3.x + Jupyter Notebook / Quarto: entorno de desarrollo interactivo que permite la integración de código, narrativa y visualizaciones (soporte para *literate programming*).
- GitHub: repositorio colaborativo para el control de versiones del proyecto final; el historial de commits forma parte de la evidencia de aprendizaje.
- Plataformas de IA generativa (ChatGPT, Copilot, Gemini, etc.): uso orientado por guías éticas de uso institucional; los estudiantes documentan y analizan críticamente sus interacciones.
- Simuladores y visualizadores: Python Tutor para la visualización de referencias y pila de ejecución; simuladores de ciclo fetch-decode-execute y de estados de proceso del SO.
- Material de elaboración propia (disponible en el campus): 'Arquitecturas de Computadoras: Von Neumann vs. Harvard', 'Lenguajes de Máquina y Evolución de la Programación', 'Tipos de Datos Básicos en Computadoras', 'Funciones del Sistema Operativo y Administración de Archivos', 'Fundamentos de Programación: Algoritmia y Python'.

9. Conclusión

Este plan de labor docente promueve el aprendizaje de Algoritmia y Programación desde una perspectiva holística y espiral, donde los conceptos de Arquitectura, Sistemas Operativos y Lenguajes de Programación se presentan como fundamentos para entender y articular los desafíos propios de la programación. La organización semanal en clases teóricas, prácticas de laboratorio y actividades de extracursado, articulada con el sistema de competencias codificadas, permite una alineación constructiva clara entre objetivos, actividades de aprendizaje y criterios de evaluación.

Se pone especial énfasis en la integración teórica y práctica, el trabajo colaborativo, el análisis crítico del uso de la tecnología y la inteligencia artificial, y la construcción de una actitud activa, autónoma y

éticamente responsable frente a la informática — condiciones necesarias para el ejercicio profesional de la ingeniería en el siglo XXI.